```python
import pandas as pd
import numpy as np
import torch
from datasets import Dataset, DatasetDict, ClassLabel, Features
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    pipeline
)
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
```

```
2025-11-16 14:46:52.212563: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1763304412.384699      18 cuda_dnn.cc:8310] Unable to register cuDNN facto
E0000 00:00:1763304412.433084      18 cuda_blas.cc:1418] Unable to register cuBLAS fac
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
```

```python
import transformers
transformers.__version__
```

```
'4.53.3'
```

```
df = pd.read_csv("/kaggle/input/legal-indian-contract-clauses-dataset/legal_contract_c
```

```
TEXT_COLUMN = "clause_text"
LABEL_COLUMN = "risk_level"
```

```
print("Step 2: Creating label mappings ... ")

# Get all unique labels
labels = df[LABEL_COLUMN].unique()
labels.sort() # Sort for consistency

# Create mappings between text labels and integer IDs
label2id = {label: i for i, label in enumerate(labels)}
id2label = {i: label for i, label in enumerate(labels)}
num_labels = len(labels)

# Add integer 'label' column to the DataFrame
df['label'] = df[LABEL_COLUMN].map(label2id)
```

```
Step 2: Creating label mappings ...
```

```
# --- 4. Convert to Hugging Face Dataset ---
print("Step 3: Converting to Hugging Face Dataset ... ")
dataset = Dataset.from_pandas(df)

# --- NEW: Cast 'label' column to ClassLabel ---
# This is required for stratification to work.
# It tells the 'datasets' library that 'label' is a category, not just an integer.
print("Step 3a: Casting 'label' column to ClassLabel for stratification ... ")

# Recreate the list of label names in the correct integer order
# (Uses the 'id2label' dictionary we created in Step 3)
label_names_in_order = [id2label[i] for i in range(num_labels)]

# Create the ClassLabel feature
class_label_feature = ClassLabel(names=label_names_in_order)

# Cast the 'label' column in the dataset
dataset = dataset.cast_column("label", class_label_feature)

print(f"Dataset features after casting: {dataset.features}")
# --- END NEW ---


# Split the dataset (e.g., 80% train, 20% test)
# This line will now work correctly
```

```
print("Step 3b: Splitting dataset ... ")
train_test_split = dataset.train_test_split(test_size=0.2, seed=42, stratify_by_colum

dataset_dict = DatasetDict({
    'train': train_test_split['train'],
    'test': train_test_split['test']
})

print(f"Training data: {len(dataset_dict['train'])} examples")
print(f"Test data: {len(dataset_dict['test'])} examples")
```

```
Step 3: Converting to Hugging Face Dataset ...
Step 3a: Casting 'label' column to ClassLabel for stratification ...
```

```
Casting the dataset:   0%|          | 0/9447 [00:00<?, ? examples/s]
```

```
Dataset features after casting: {'clause_text': Value('string'), 'clause_type': Value(
Step 3b: Splitting dataset ...
Training data: 7557 examples
Test data: 1890 examples
```

```
print("\nStep 4: Loading model and tokenizer (nlpaueb/legal-bert-base-uncased) ... ")
model_name = "nlpaueb/legal-bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=num_labels,
    id2label=id2label,
    label2id=label2id
)
```

```
Step 4: Loading model and tokenizer (nlpaueb/legal-bert-base-uncased) ...
```

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
```

```
config.json: 0.00B [00:00, ?B/s]
```

```
vocab.txt: 0.00B [00:00, ?B/s]
```

```
special_tokens_map.json:   0%|          | 0.00/112 [00:00<?, ?B/s]
```

```
pytorch_model.bin:   0%|          | 0.00/440M [00:00<?, ?B/s]
```

```
Some weights of BertForSequenceClassification were not initialized from the model chec
You should probably TRAIN this model on a down-stream task to be able to use it for pr
```

```
print("\nStep 5: Tokenizing dataset...")

def tokenize_function(examples):
    # This tokenizes the text, padding and truncating
    return tokenizer(examples[TEXT_COLUMN], padding="max_length", truncation=True)

# Apply tokenization to the entire dataset (this will be fast)
tokenized_datasets = dataset_dict.map(tokenize_function, batched=True)

# Remove the original text columns, as the model only needs token IDs
columns_to_remove = [TEXT_COLUMN, LABEL_COLUMN]
if "__index_level_0__" in tokenized_datasets["train"].column_names:
    columns_to_remove.append("__index_level_0__")

tokenized_datasets = tokenized_datasets.remove_columns(columns_to_remove)
tokenized_datasets.set_format("torch")

print("Tokenization complete.")
```

```
Step 5: Tokenizing dataset...
```

```
Map:   0%|          | 0/7557 [00:00<?, ? examples/s]
```

```
model.safetensors:   0%|          | 0.00/440M [00:00<?, ?B/s]
```

```
Map:   0%|          | 0/1890 [00:00<?, ? examples/s]
```

```
Tokenization complete.
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    acc = accuracy_score(labels, predictions)
    f1 = f1_score(labels, predictions, average='weighted')
    precision = precision_score(labels, predictions, average='weighted')
    recall = recall_score(labels, predictions, average='weighted')

    return {
```

```
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
    }
```

```
# --- 8. Set Training Arguments & Train ---
print("\nStep 6: Setting training arguments...")

# Define the output directory for the fine-tuned model
model_output_dir = "./legal_bert_finetuned_risk"

training_args = TrainingArguments(
    output_dir=model_output_dir,
    learning_rate=2e-5,
    per_device_train_batch_size=8,  # Adjust down (4, 8) if you get CUDA OOM errors
    per_device_eval_batch_size=8,
    num_train_epochs=3, # 3 epochs is a good starting point
    weight_decay=0.01,
    load_best_model_at_end=True,
    push_to_hub=False,
    report_to="none", # Disables wandb logging
    eval_strategy="epoch",
    save_strategy="epoch",
)

# Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)
```

```
Step 6: Setting training arguments...
```

```
/tmp/ipykernel_18/3109696080.py:22: FutureWarning: `tokenizer` is deprecated and will
  trainer = Trainer(
```

```
print("\nStep 7: Starting model training (this may take a while)...")
# Start training!
trainer.train()

print("Training finished.")

# Save the final model and tokenizer
trainer.save_model(model_output_dir)
```

```
tokenizer.save_pretrained(model_output_dir)
print(f"Model saved to {model_output_dir}")
```

```
Step 7: Starting model training (this may take a while)...
```

[2835/2835 29:12, Epoch 3/3]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.453900	0.213300	0.940741	0.940900	0.941751	0.940741
2	0.138200	0.218354	0.948148	0.948154	0.948161	0.948148
3	0.059800	0.244474	0.948677	0.948705	0.948776	0.948677

```
Training finished.
Model saved to ./legal_bert_finetuned_risk
```

```
print("\nStep 8: Using the fine-tuned model for inference...")

# Load the fine-tuned model from the directory where we saved it
finetuned_model_path = "./legal_bert_finetuned_risk"
risk_classifier = pipeline(
    "text-classification",
    model=finetuned_model_path,
    # Use id 0 if no GPU, or 0, 1, etc. for specific GPUs
    device=0 if torch.cuda.is_available() else -1
)

print("Fine-tuned model loaded into a pipeline.")
```

```
Device set to use cuda:0
```

```
Step 8: Using the fine-tuned model for inference...
Fine-tuned model loaded into a pipeline.
```

```
test_clause = "Indemnification. The Contractor (the 'Indemnitor') agrees to indemnify,

result = risk_classifier(test_clause, return_all_scores=True)
print(f"\nTest Clause: '{test_clause[:100]}...'\n")
print(f"Prediction: {result}")

# Test with a low-risk clause
test_clause_low = "This settlement agreement is reached amicably between both parties.
result_low = risk_classifier(test_clause_low)
print(f"\nTest Clause: '{test_clause_low}'\n")
print(f"Prediction: {result_low}")
```

```
/usr/local/lib/python3.11/dist-packages/transformers/pipelines/text_classification.py:
  warnings.warn(
```

```
Test Clause: 'Indemnification. The Contractor (the 'Indemnitor') agrees to indemnify,

Prediction: [[{'label': 'high', 'score': 0.7944066524505615}, {'label': 'low', 'score'

Test Clause: 'This settlement agreement is reached amicably between both parties.'

Prediction: [{'label': 'medium', 'score': 0.6971290707588196}]
```